

PYTHON O CÓDIGO DA
RESISTÊNCIA CONTRA O IMPÉRIO

MERGULHANDO NO BASH



LISTA DE FIGURAS

Figura 1 – Que a Força do Bash esteja com você	6
Figura 2 – <i>Shell Bash</i>	8
Figura 3 – Resultado do exercício 1.....	13
Figura 4 – Resultado do exercício 2.....	13

EMANIP

LISTA DE QUADROS

Quadro 1 – Comandos de navegação no Bash.....	8
Quadro 2 – Expressões úteis para caminhos no Bash.....	10
Quadro 3 – Expressões úteis utilizando chaves no Bash.....	10
Quadro 4 – Exercício 3 em seis partes	14
Quadro 5 – Resposta do exercício 3 em seis partes.....	14

EMANIP

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Chamada do comando manual para mkdir	8
Comando de prompt 2 – Dinâmica de filtros em linha de comando	11
Comando de prompt 3 – Exemplo de redirecionador de saída	11
Comando de prompt 4 – Exemplo de redirecionador de entrada	11
Comando de prompt 5 – Exemplo uso o redirecionador <i>pipeline</i>	12
Comando de prompt 6 – Resultado do exercício 1	13
Comando de prompt 7 – Resultado do exercício 2	13

EXEMPLO

SUMÁRIO

1 MERGULHANDO NO BASH	6
1.1 Bourne Again Shell.....	6
2 EXPANSÕES	9
3 REDIRECIONAMENTOS	11
4 QUE TAL UM POUCO DE PRÁTICA?	12
REFERÊNCIA	15

EMANIP

1 MERGULHANDO NO BASH

Existe um oceano de possibilidades que o *Shell* Linux, mais especificamente o *Bash*, pode nos proporcionar.

Preparado para mergulhar neste oceano?

1.1 Bourne Again Shell

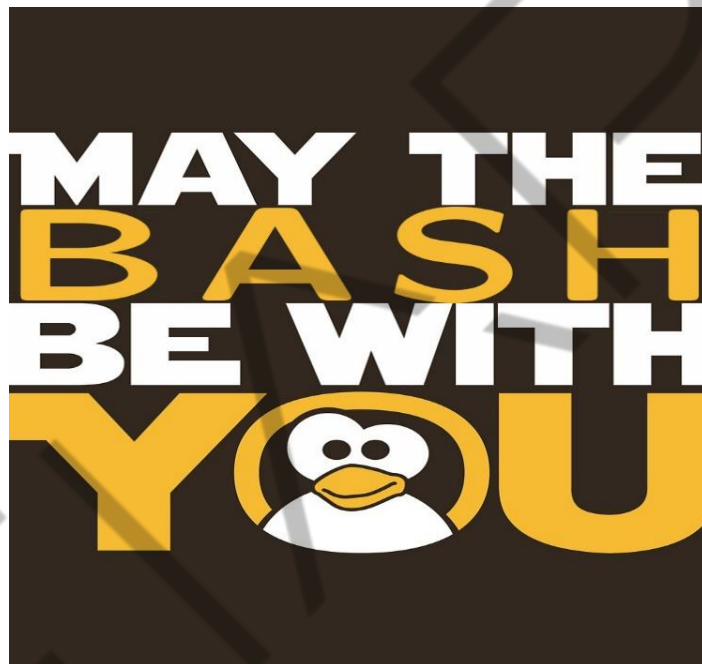


Figura 1 – Que a Força do Bash esteja com você
Fonte: Google Imagens (2019)

Amplamente utilizado no universo Linux, o Shell Bash muitas vezes é conceituado como um simples interpretador de comandos, porém, de acordo com Silva (2018), o *Bourne Again Shell* vai muito além disso, pois em ambientes UNIX o conceito de arquivo é amplo, ou seja, muitas vezes é visto como uma sequência de bytes que obedece a um padrão.

Outro fato interessante é que, para o UNIX (e obviamente para o Linux, seu derivado), a extensão do arquivo pouco importa. O Sistema Operacional o reconhece por conta do padrão de *bytes* que o compõe, muitas vezes baseado no *header* (cabeçalho) dessa sequência.

Silva (2018) separa os arquivos em três tipos:

- Arquivos de texto puro, os quais contêm apenas caracteres legíveis do tipo ASCII.
- Arquivos executáveis, em que o conteúdo muitas vezes não é composto por caracteres legíveis.
- Arquivos de Diretório ou, como geralmente são chamados, “pasta”. São estruturas lógicas que armazenam outros arquivos (qualquer tipo, até outros diretórios).

Ainda de acordo com Silva (2018), o conceito da árvore de diretórios é abstraído em sistemas derivados de UNIX, pois ainda que os dados não sejam armazenados dessa forma, os diretórios são hierarquizados, ou seja, todos têm um mesmo pai, assim, se dois diretórios tiverem o “mesmo pai”, são irmãos. Tudo tem seu começo no diretório raiz que é o “/”.

Para navegar na árvore de diretórios sem precisar conhecer ou digitar o caminho completo do arquivo, existe o conceito do “diretório atual” para que você possa referenciar o arquivo relativo ao diretório e o arquivo referenciado absoluto (a partir da raiz). O caminho deve começar com uma barra “/”, sendo que os diretórios também serão separados por ela, ou seja, para que o caminho seja válido, é necessário manter a sequência da conexão existente na árvore de diretórios.

Dessa forma, cada diretório tem uma conexão com o diretório pai “..” e o próprio “.” para que você possa acessar qualquer diretório a partir de um outro. Se o diretório atual for profundo, de um galho de árvore, será acessado a partir da raiz inicial. Acessar o diretório desejado por um caminho absoluto pode ser mais rápido do que retornar vários diretórios usando “..”.

Conforme apresentado por Silva (2018), além da estrutura de diretórios e do diretório atual no UNIX, cada usuário possui um diretório-base (**/home**). Ele separa arquivos do sistema, programas executáveis e outros arquivos dos diretórios pessoais do usuário. Normalmente, os usuários podem usar um atalho para que possam sempre se referir à sua home, utilizando o caractere TIL(~).

/home/usuario/Documentos

../media/dispositivo

~/Downloads

Para navegar na árvore de diretórios, podemos utilizar os comandos abaixo:

Nome do comando	Função
pwd	Retorna o diretório atual
ls	Lista os arquivos de um diretório
cd	Muda o diretório atual
mkdir	Cria um novo diretório

Quadro 1 – Comandos de navegação no Bash
Fonte: Elaborado pelo autor (2019)

Existem opções para visualizar metadados e anexar arquivos ocultos cujos nomes de arquivos estão marcados no início com um ".". Tanto o comando **ls** quanto o **cd** aceitam os caminhos absolutos e relativos.

Para obter mais informações sobre opções e parâmetros de comando, consulte as páginas do manual, acessando da seguinte maneira: **man [comando]**. Por exemplo:

```
$ man mkdir
```

Comando de prompt 1 – Chamada do comando manual para mkdir
Fonte: Elaborado pelo autor (2019)

Dica: Você pode usar o recurso de preenchimento automático para evitar digitações desnecessárias! Pressione o TAB uma única vez quando existir apenas uma opção possível. Ao pressionar o TAB duas vezes, serão exibidas todas as opções possíveis para concluir o comando e inserir nomes e diretórios de arquivos existentes e acessíveis no caminho (não se esqueça que os comandos também são arquivos, como, por exemplo, o comando Bash, que está localizado em **/bin**).



Figura 2 – Shell Bash
Fonte: Google Imagens (2019)

2 EXPANSÕES

É bem verdade que a navegação por meio dos diretórios de um sistema Linux não é uma tarefa simples, principalmente porque nem sempre conseguimos manter um “mapa-mental” dos subdiretórios presentes em nosso sistema. E são em momentos como esse que vemos os poderes do *Bash* em ação, pois o uso de caracteres especiais (meta-caracteres ou “curingas”) possibilita expandir nossas buscas.

De acordo com Silva (2018), o asterisco (*) é capaz de substituir qualquer sequência de caracteres (e de qualquer tamanho), assim como o ponto de interrogação (?) substitui como curinga um único caractere. Para um exemplo hipotético, suponha que tenhamos três arquivos: **c16d8a16.txt**, **c4c4a316.txt** e **c128d2a8.txt**, e que existam, também, uma infinidade de arquivos com nomes variados de modo que **ls** não é a forma mais rápida de identificar arquivos (aparecem muitos resultados). Se usarmos o asterisco, torna-se possível listar um arquivo sabendo apenas parte de seu nome. Quer um exemplo? O comando “**ls c16***” retornaria apenas o arquivo **c16d8a16.txt** (supondo que naquela infinidade de arquivos nenhum outro arquivo do diretório corrente comece com “**c16**”).

Embora o comando “**ls c1?d**” retorne o mesmo arquivo **c16d8a16.txt**, ao utilizar o “*” (asterisco) no lugar do “?” (ponto de interrogação), o comando retornaria tanto o arquivo “**c16d8a16.txt**” quanto o “**c128d2a8.txt**”.

Imagina agora que você deseja exibir o conteúdo de todos os arquivos que terminarem com a extensão “**.txt**”; para isso, o comando **cat** certamente deve ser empregado. Seja porque o usuário não sabe o nome de todos os arquivos, seja porque não queira digitar todos. Ele pode resolver o problema com um simples comando: **cat *.txt**.

Ainda de acordo com Silva (2018), os curingas e expressões regulares podem ser usados em comandos como o **mv** (mover), **cp** (copiar) e o **rm** (remover). A dica essencial para dominar o *Bash*, no entanto, é praticar! Que tal criar um diretório para testar comandos com “*” e “?”, em especial com o comando **rm** (remover)? Mas tome muito cuidado com o uso de “.*”, lembre dos diretórios **.** e **..** antes de executar

comandos destrutivos. Não parece, mas o **rm** (especialmente com parâmetros especiais) pode ser muito perigoso!

Com o uso dos colchetes (“[” e “]”) é possível criar curingas mais específicos para a expansão de caminhos, por exemplo, a expansão ***.[abc]** funciona para todos os 3 casos: ***.a**, ***.b** e ***.c**.

Expressão	correspondente à
[b-e;!]	Qualquer letra minúscula de “b” a “e”, “.”, “,” e “!”
[!b-e]	Qualquer dígito, exceto letras minúsculas de “b” a “e”
[A-Z0-9]	Qualquer letra maiúscula de “A” a “Z” e qualquer número de um dígito
[a-df-i]	a, b, c, d, f, g, h, i
[abc-]	“a”, “b”, “c”, ou “-”

Quadro 2 – Expressões úteis para caminhos no Bash
Fonte: Elaborado pelo autor (2019)

A expansão de chave permite estender caminhos que não existem, portanto, dependendo da situação, você pode obter respostas de erro do comando usado.

Os comandos em *Bash* geram mensagens de erro muito intuitivas. Portanto, antes de pesquisar no Google, leia as mensagens de erro e procure nas páginas de manual, se necessário, pois, na maioria dos casos, isso resolverá seu problema. Se tiver dúvidas sobre o resultado de uma expansão, use o comando `echo` para verificar se ela produzirá os resultados esperados antes de usá-la em comandos que possam danificar ou mover arquivos importantes. Não esqueça que existe diferença entre expansão de chave entre colchetes!

Expressão	Resultando obtido
cat colaborador.{c,h}	Conteúdo dos arquivos colaborador.c e colaborador.h.
ls gravacao0{1..5}.mp3	Gravacao01.mp3, gravacao016.mp3, gravacao03.mp3, gravacao04.mp3 e gravacao05.mp3.
touch 201{7..9}/exemplo{1..4}.txt	Criar os arquivos em branco exemplo1.txt, exemplo16.txt, exemplo3.txt e exemplo4.txt nos diretórios 2017, 2018 e 2019 (12 arquivos no total!).
echo grupo{A..D}	grupoA, grupoB, grupoC e grupoD.
echo grupo{A..G..2}	grupoA, grupoC, grupoE, grupoG (pulando letras, de dois em dois!).

Quadro 3 – Expressões úteis utilizando chaves no Bash

Fonte: Elaborado pelo autor (2019)

3 REDIRECIONAMENTOS

Segundo Silva (2018), o redirecionamento é o próximo passo na evolução dos comandos de script. Expansões são úteis, mas nem todos os comandos podem resolver todos os problemas. Uma boa abstração para entender a funcionalidade de redirecionamento do Bash é tratar os comandos como filtros. O comando recebe entrada e produz saída. Até esse ponto, a entrada é sempre lida pelo teclado (o chamado periférico padrão de entrada, **stdin - Standard Input**) e impressa na tela (periférico padrão de saída, **stdout - Standard Output**), mas se o comando puder ser usado como filtro, podemos usar a saída de um comando e redirecioná-la para a entrada de outro comando, combinando vários comandos para obter a saída desejada, veja este exemplo:

```
entrada -> Filtro1 -> Filtro2 -> ... -> FiltroN -> saída
```

Comando de prompt 2 – Dinâmica de filtros em linha de comando

Fonte: Elaborado pelo autor (2019)

O caractere que é utilizado para redirecionamentos de entrada é “<” e, “>” é usado para redirecionamentos de saída. Então, o seguinte exemplo redireciona a saída do cat para o arquivo teste.txt:

```
$ cat texto.txt > teste.txt
```

Comando de prompt 3 – Exemplo de redirecionador de saída

Fonte: Elaborado pelo autor (2019)

Você também pode redirecionar entradas para comandos. Suponha que exista um programa que leia a entrada padrão. Se a entrada de teste for muito grande, você pode inserir um arquivo de redirecionamento como este:

```
$ ./programa < entrada1.txt
```

Comando de prompt 4 – Exemplo de redirecionador de entrada

Fonte: Elaborado pelo autor (2019)

Observação: “./” é a maneira correta de se executar um arquivo executável em ambientes UNIX (tente “./ls” no diretório “/bin”, por exemplo).

Como mencionado anteriormente, a última ferramenta que podemos utilizar programas e comandos como filtros é o *pipe* (`|`). Os pipelines eliminam a necessidade de arquivos intermediários e vinculam diretamente a saída de um programa à entrada de outro da seguinte maneira:

```
# ls /dev | grep std | tee arq1.txt arq16.txt arq3.txt
```

Comando de prompt 5 – Exemplo uso o redirecionador *pipeline*
Fonte: Elaborado pelo autor (2019)

O resultado deste comando mostra 3 linhas, cada uma contendo um nome de arquivo: "**stdin**", "**stdout**" e "**stderr**". Por convenção, **stderr** é onde os avisos e erros do programa (como *logs*) são impressos. O comando `ls` está listando os arquivos no diretório **/dev**. O comando `grep` está selecionando apenas as linhas de resultados que contêm a *substring* "**std**" passada como parâmetro. O comando `tee` está redirecionando a entrada para todos os arquivos passados como parâmetros e também como saída padrão.

Conforme Silva (2018), comumente se usa o pipeline com o `grep` para filtrar dados dentro da saída do `ls`, por exemplo. Também são muito utilizados os comandos `cut` e `tr` com o intuito de filtrar linhas e substituir expressões na saída do `cat`, por exemplo. Você também poderia substituir os comandos apresentados por processadores de texto (ou editores, como alguns autores conceituam) como `sed` e `awk`, com o intuito de identificar padrões e extrair ou substituir sentenças na saída de outros comandos e/ou arquivos de texto.

4 QUE TAL UM POUCO DE PRÁTICA?

Que tal colocar os conhecimentos adquiridos em prova? Convidamos você a realizar alguns exercícios para treinar os conhecimentos adquiridos até aqui com o intuito de massificá-los (lembre-se: quando o treino é difícil, a guerra se torna fácil).

1. Utilizando um único comando **mkdir** e expansão de chaves, crie os 7 diretórios com os seguintes nomes:

- a) 3 diretórios para "deptoJUR", "deptoRH" e "deptoTI";
- b) 3 diretórios para "deptoA", "deptoB" e "deptoC"; e

c) 1 diretório “colaboradores”.

Não se esqueça de colocar tudo dentro de um diretório seguro para testes.

E então?

Vamos à resposta?

```
# mkdir depto{{A..C},{JUR,RH,TI}} colaboradores
```

Comando de prompt 6 – Resultado do exercício 1

Fonte: Elaborado pelo autor (2019)

Veja em funcionamento na Figura “Resultado do exercício 1”.

```
hpoyatos@gentoo ~/praticaExpansoes $ mkdir depto{{A..C},{JUR,RH,TI}} colaboradores
hpoyatos@gentoo ~/praticaExpansoes $ ls
colaboradores deptoA deptoB deptoC deptoJUR deptoRH deptoTI
hpoyatos@gentoo ~/praticaExpansoes $ _
```

Figura 3 – Resultado do exercício 1

Fonte: Elaborado por Henrique Poyatos (2019)

2. Dentro do diretório colaboradores, crie arquivos de nome de colaboradores no formato **colLnotaN** substituindo a letra “L” de linguagem por: PHP, JAVA, PYTHON e, para cada linguagem, um número de matrícula (letra “N”) que vá de 0 a 9 e NULL.

Exemplo de arquivo: **colJAVAmatrNULL**

Preparado para a resposta?

```
# touch colaboradores/col{PHP,JAVA,PYTHON}matr{{0..9},NULL}
```

Comando de prompt 7 – Resultado do exercício 2

Fonte: Elaborado pelo autor (2019)

Ficando assim:

```
hpoyatos@gentoo ~/praticaExpansoes $ touch colaboradores/col{PHP,JAVA,PYTHON}matr{{0..9},NULL}
hpoyatos@gentoo ~/praticaExpansoes $ ls colaboradores/
colJAVAmatr0 colJAVAmatr6 colPHPmatr1 colPHPmatr7 colPYTHONmatr2 colPYTHONmatr8
colJAVAmatr1 colJAVAmatr7 colPHPmatr2 colPHPmatr8 colPYTHONmatr3 colPYTHONmatr9
colJAVAmatr2 colJAVAmatr8 colPHPmatr3 colPHPmatr9 colPYTHONmatr4 colPYTHONmatrNULL
colJAVAmatr3 colJAVAmatr9 colPHPmatr4 colPHPmatrNULL colPYTHONmatr5
colJAVAmatr4 colJAVAmatrNULL colPHPmatr5 colPYTHONmatr0 colPYTHONmatr6
colJAVAmatr5 colPHPmatr0 colPHPmatr6 colPYTHONmatr1 colPYTHONmatr7
hpoyatos@gentoo ~/praticaExpansoes $ _
```

Figura 4 – Resultado do exercício 2

Fonte: Henrique Poyatos (2019)

3. Copie os arquivos do diretório colaboradores para o diretório correspondente, de acordo com a tabela a seguir:

Exercício	Arquivos	Copiar para
3.1	Colaboradores de PHP com matrícula diferente de NULL	deptoA
3.2	Colaboradores de JAVA com matrícula diferente de NULL	deptoB
3.3	Colaboradores de PYTHON com matrícula diferente de NULL	deptoC
3.4	Colaboradores de PHP, JAVA, PYTHON com matrícula de 0 até 3	deptoJUR
3.5	Colaboradores de PHP, JAVA, PYTHON com matrícula de 4 até 6	deptoRH
3.6	Colaboradores de PHP, JAVA, PYTHON com matrícula de 7 até 9	deptoTI

Quadro 4 – Exercício 3 em seis partes

Fonte: Elaborado pelo autor (2019)

Está pronto?

Exercício	Resultado
3.1	<code>cp colaboradores/colPHPmatr[!A-Z] deptoA/</code>
3.2	<code>cp colaboradores/colJAVAmatr[0-9] deptoB/</code>
3.3	<code>cp colaboradores/colPYTHONnota[0-9] deptoC/</code>
3.4	<code>cp colaboradores/col{PHP,JAVA,PYTHON}matr[0-3] deptoJUR/</code>
3.5	<code>cp colaboradores/col{PHP,JAVA,PYTHON}matr[4-6] deptoRH/</code>
3.6	<code>cp colaboradores/col{PHP,JAVA,PYTHON}matr[7-9] deptoTI/</code>

Quadro 5 – Resposta do exercício 3 em seis partes

Fonte: Elaborado pelo autor (2019)

Experimente criar cenários como estes apresentados aqui para que você consiga treinar sua prática no *Bash*. Os sistemas Linux são amplamente usados no cenário de *Cyber Security* e saber manuseá-los é mais que uma necessidade, é uma obrigação para um profissional de Defesa Cibernética.

REFERÊNCIA

SILVA, G. B. B. da. **Tutorial do Bash**. Paraná: Universidade Federal do Paraná, 2018.

EMEND